



Architecting The
NG-ine Room





From Idea to MVP: Building AI-Powered Angular Apps with Firebase

Ayman Issa

Software Engineer & Developer Advocate

Meet De{c0}dE

- Paste code → get a structured breakdown
- Explanation, issues, refactor, quality score
- Real app, real AI — not a mockup

Agenda

- The problem, the approach, and why this stack
- Every architecture decision — including the ones that didn't work
- Building it live: auth, data, and the AI layer
- A live demo, end to end
- Build it yourself — a full codelab, together

The Problem

Most product ideas don't fail because they were bad ideas. They fail because turning "I have an idea" into "I have something people can actually try" takes too long — and momentum, budget, and belief all run out before you find out if the idea was worth building.

This talk is about closing that gap — validating an idea by actually shipping, not sketching it.

The Approach

Not heavy implementation, and not a generic tutorial. This is about architecture, workflow, and the decisions that actually determine whether an idea becomes a product — the calls you make about structure, backend, and where AI genuinely earns its place in the app, not just where it's trendy to bolt on.

Angular for the UI, Firebase for auth and data, AI where it adds real value — validated fast, iterated with confidence.

“

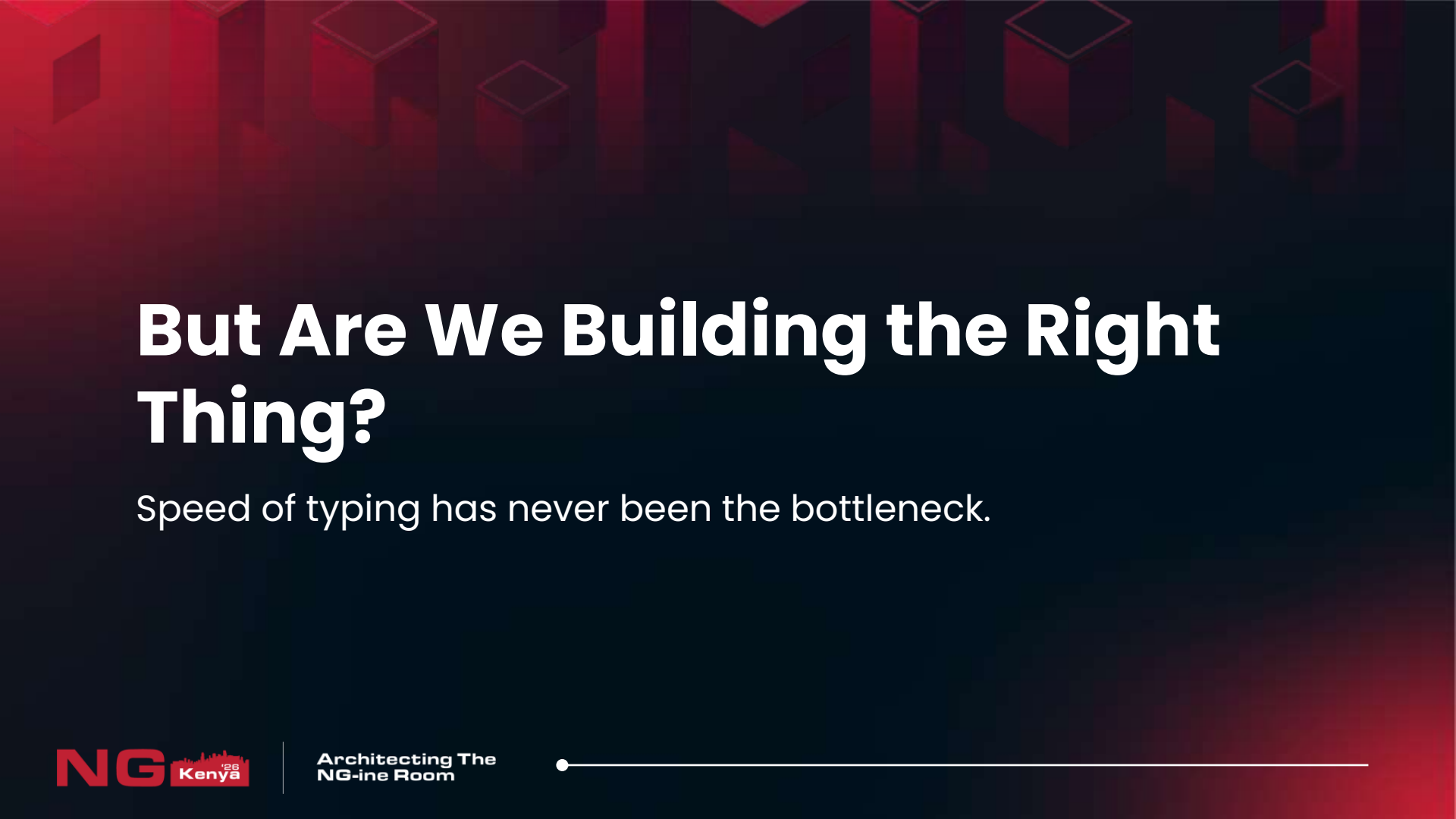
Angular + Firebase + AI — used deliberately,
not by default.

The Stack

- Angular — scalable UI, built for rapid iteration
- Firebase Authentication — real auth in minutes, not days
- Cloud Firestore — real-time data with zero backend code
- Hugging Face Inference API — the AI layer, running for real

We Can Build Faster Than Ever

- AI-assisted coding
- Component libraries
- One-command deploys
- The barrier to shipping has never been lower



But Are We Building the Right Thing?

Speed of typing has never been the bottleneck.

Most MVP Time Goes Into Infrastructure Decisions

None of these validate whether anyone wants the product.

Which auth provider?

Which database?

Which hosting?

How do I secure it?

How do I scale it?

How do I deploy it?

Choosing the Frontend Library

Not a framework war — a contextual fit question for this MVP.

Why Angular

- Fewer decisions — routing, DI, forms, signals: built in
- Strong file and feature conventions
- Standalone components by default
- Trade-off: steeper initial curve than React's minimal one

Why Not React

- A UI library, not an application framework
- Routing, DI, everything else — you assemble it yourself
- Its edge: ecosystem flexibility, exact architecture you want
- Better fit for teams with strong existing opinions

Why Not Next.js

- Powerful — for problems this MVP doesn't have
- SSR, server components, hybrid rendering: solving what we don't need
- Stronger choice for content-heavy, SEO-driven, hybrid-rendering products

Why Firebase

- Validate the product — don't rebuild infrastructure
- Auth + database + hosting + AI layer: one platform, not five vendor decisions

The Actual Stack



Angular 22

Scalable UI



Firebase

Auth + Firestore



Hugging Face

Inference API

Login / Register



Auth Guard

blocks access if not signed in



Workspace (Angular)



Firebase Auth
session

Cloud Firestore
profile · settings · history

Hugging Face API
analyze code



Rendered Result in UI

Angular App Architecture

- One component per page
- Shared App Shell — reused across every signed-in page
- A real spec under deadline pressure — worth naming honestly, not gospel

landing

login

register

workspace

settings

profile

brand

privacy

terms

Authentication Architecture

- Angular → Firebase Authentication → Google or email
- Session handled automatically
- Better password UX: feedback while typing, not after submit

```
// Email/password login
async login(email, password) {
  await signInWithEmailAndPassword(...)
}

// Google sign-in – one click, no password
async loginWithGoogle() {
  await signInWithPopup(auth, new GoogleAuthProvider())
}
```

The Data Model

- Every user gets their own tree
- Ownership is encoded in the path itself — not a permissions table

```
users/{uid}
```

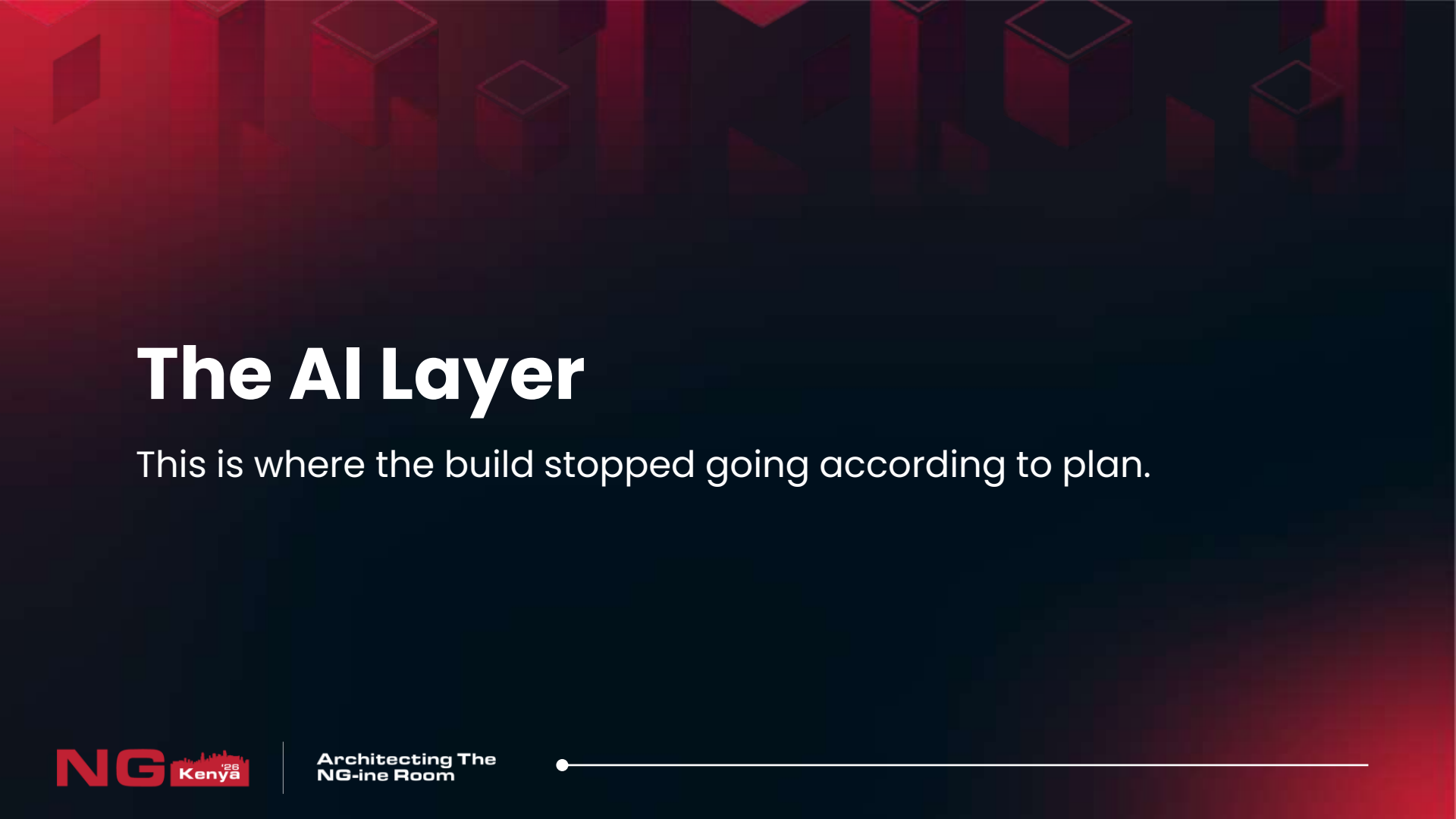
```
email, displayName, createdAt
```

```
analyses/{analysisId}
```

```
codeSnippet, language, summary
```

```
analysis (the full structured result)
```

```
createdAt
```



The AI Layer

This is where the build stopped going according to plan.

Plan A: Firebase AI Logic

- Couldn't use Cloud Functions — needs the paid Blaze plan
- So the plan became Firebase AI Logic — the recommended path
- (Broke student-conference-budget rule #1: no card on file)

What Actually Happened

- Persistent 403: “Your project has been denied access”
- Across multiple fresh projects, multiple fresh keys
- Not just us — a widespread report from other developers

Google AI Developers Forum

discuss.ai.google.dev — same 403, same wording, different developers

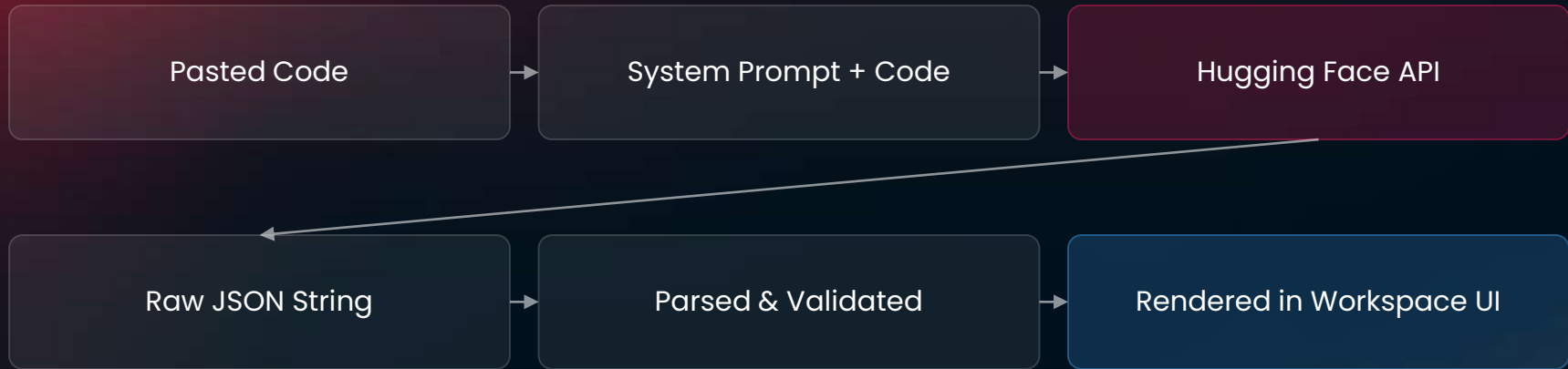
The Pivot to Hugging Face

- Free, open-model inference API — no billing account needed
- Same client-side call shape — swap the SDK, not the architecture
- Lesson: a Google-recommended model doesn't guarantee it works for your account

Response Contracts

- Don't ask AI for “some JSON”
- An AI response is a contract, not an uncontrolled process
- Enforce the shape — don't hope for it

```
{  
  "language", "summary", "explanation": string,  
  "steps": string[],  
  "issues": [{ severity, title, description }],  
  "suggestions", "refactorBefore", "refactorAfter": string,  
  "score": { maintainability, grade, complexity, readability },  
  "followUps": string[]  
}
```



Conversation, Not One-Shot

- Submit code, get an analysis, ask a follow-up
- Follow-ups persist as messages in a growing thread
- Saved to Firestore — like real chat history

Skeletons, Not Blank Screens

- Loading is part of the product, not an afterthought
- No fake progress percentages
- Just an honest “this is coming”

Transparent Fallback

- When the model's output can't be parsed, say so
- Show the raw response — not a generic error
- The user sees what actually happened, not a lie about what didn't

Rules & Guards

- Not route-hiding — Firebase Auth is the real identity source
- Firestore security rules enforce ownership via the path itself
- Real route guards on every private page

Live Demo

What I Learned

- Model names get deprecated — “recommended” $\neq$ “works for your account”
- Newest library version $\neq$ automatically compatible with your framework version

Most Side Projects Never Ship

- Most never leave the ideation stage
- You can go idea → MVP fast, without cutting corners on feel
- The thesis: Angular (UI) + Firebase (backend) + Hugging Face (intelligence)

Defer the Expensive Part

- Ship the UI on mock data first
- Prototype with cheaper alternatives — Hugging Face instead of paid Cloud Functions

A Design System Multiplier

- One set of tokens — color, type, spacing
- Every new page: minutes, not hours

Structure = Reliability

- A fixed AI contract turns AI from a hazard into a data source
- Structured output is what makes it a UI you can build on

Still an MVP: What's Next

- A real backend proxy — Cloud Functions, keys off the client
- Rate limiting, so one user can't exhaust the quota
- Shareable analyses — send a result, not just a screenshot

Your Turn

Don't watch it — let's build it together. Any questions before we start?



Questions





Ayman Issa

Software Engineer & Developer
Advocate



@aymanissa
_dev



aymanissa-dev

